Kasi Yedumati · Full Stack Training

Module 04 — Exception Handling

Phase: Advanced Python

Previous: OOP Part 4 — Abstract Classes & Interfaces
Next: File Handling

Module Outcome

Catch runtime failures by type, recover or fail with a clear message, and name business errors so callers can handle them without a traceback on the screen.

By the end of this module you will be able to:

- Tell syntax errors, runtime exceptions, and logical errors apart
- Use `try` / `except` for a risky attempt and named recovery
- Catch several types (tuple or stacked `except`)
- Use `else` for the success path and `finally` for cleanup
- `raise` built-in and custom exceptions; never use a bare `except:`

1. Why This Concept Matters

A clerk types an age. `"abc"` is not a number. Without handling, Python prints a traceback and the whole program **stops**. The desk screen should say “age must be an integer” and let them try again.

Python has **three** kinds of problems. Only one of them is what this module handles.

Kind	When	Traceback?	`try / except` ?
Syntax error	Before the program runs	Yes (parse time)	No — edit the source
Runtime error / exception	While running	Yes if unhandled	Yes — this module
Logical error	Runs, wrong result	No	No — debug / test

Syntax error — Python cannot even read the file. Missing `:`, `if pin = 1234` instead of `==`. `try` in that same file never runs.

Runtime error (exception) — The program started, then hit a problem it cannot solve: `int("abc")`, divide by zero, missing file. Python **raises** an object with a type (`ValueError`, `ZeroDivisionError`, …) and a message. If nobody handles it, you get a traceback and the program **stops**. This module is about these.

Logical error — The program finishes. The answer is just wrong: GST 18% coded as 8%. No traceback. Tests and careful reading — not `except`.

```
price = 1000
gst = price * 0.08   # logical error: should be 0.18 — no exception
```

An ATM does not explode on a wrong PIN. It shows “Invalid PIN” and lets the user retry. That is exception handling: show a message, retry, or shut down cleanly.

2. `try` and `except`

Understand

`try` is the **risky attempt**. Only the lines that might fail belong here: `int(pin)`, split a bill, look up a dict key. Keep it short. If `try` is 40 lines, you will not know which line raised.

`except SomeType` is **named recovery**. “If this type of failure happened, do this instead of crashing.”

`except ValueError as e` binds the error object to `e` so you can print the message.

How It Works

```
raw_values = ["abc", "0", "4"]
for raw in raw_values:
    try:
        num = int(raw)
        print(100 / num)
    except ValueError:
        print("Please enter a valid integer.")
    except ZeroDivisionError:
        print("Cannot divide by zero.")
```

Expected Output:

```
Please enter a valid integer.
Cannot divide by zero.
25.0
```

Python uses the **first matching** `except`. Put specific types (`ValueError`) above a broad `Exception`.

⚠ WATCH OUT

```
# Wrong — also catches Ctrl+C (KeyboardInterrupt)
except:
    pass

# Wrong — swallows the failure; nobody knows it happened
except ValueError:
    pass

# Prefer — name the type, then recover or re-raise
except ValueError as e:
    print("Need a number —", e)
```

Do not use exceptions for everyday “is this empty?” checks. Use `if` / `else` for expected cases.

Try It

Loop `["ab12", "1234"]`. Try `int(...)` on each. Catch `ValueError` and print a digits-only message. Print `PIN accepted:` plus the number when it works.

3. Catching multiple exceptions

Understand

When two types share the **same** recovery, use a tuple: `except (TypeError, ValueError) as e:`.

When the user should do **different** things, stack `except` blocks (first match wins).

How It Works

Same recovery — one tuple:

```
def load_marks(path, files):
    if path not in files:
        raise FileNotFoundError(path)
    return int(files[path])

files = {"marks.txt": "87", "bad.txt": "eighty"}
for path in ("marks.txt", "bad.txt", "missing.txt"):
    try:
        print("Marks:", load_marks(path, files))
    except (FileNotFoundError, ValueError) as e:
        print("Could not read marks:", type(e).__name__, "—", e)
```

Expected Output:

```
Marks: 87
Could not read marks: ValueError — invalid literal for int() with base 10: 'eighty'
Could not read marks: FileNotFoundError — missing.txt
```

Different recovery — stack the handlers:

```
data = {"name": "Priya"}
try:
    age = int(data["age"])
except KeyError:
    print("age field is missing")
except ValueError:
    print("age must be a number")
```

Expected Output:

```
age field is missing
```

⚠ WATCH OUT

```
# Wrong — old Python 2 style; this is not a tuple
except ValueError, KeyError:

# Prefer
except (ValueError, KeyError) as e:
```

A lone `except Exception` at the top hides bugs. Catch narrow types first.

Try It

Given `data = {"age": "twenty"}`, use stacked `except` so a missing `"age"` prints `age field is missing` and a non-numeric value prints `age must be a number`.

---

4. `else` and `finally`

Understand

Piece	Runs when	Typical job
`try`	Always entered first	Risky work only
`except`	Matching type raised in `try`	Message, retry, fallback
`else`	`try` succeeded (no exception)	Receipt, next screen
`finally`	Always, after the others	Return the card, close, log

`else` means: "`try` fully succeeded — now do the happy-path work." An error in `else` is **not** handled by this `except`.

`finally` is cleanup that **always** runs: success, caught error, even `return` from a function. Do not put business decisions here (`if paid: ship_order()` belongs in `else`).

```
try: risky code
        |
        +-- exception, type matches --> except --> finally --> continue
        +-- exception, no match     --> finally --> traceback
        +-- no exception            --> else   --> finally --> continue
```

How It Works

```
CORRECT = 1234
for pin in ("ab12", "0000", "1234"):
    try:
        number = int(pin)
        if number != CORRECT:
            raise ValueError("Wrong PIN")
    except ValueError as e:
        print("Access denied:", e)
    else:
        print("Welcome")
    finally:
        print("Please remove your card")
    print("---")
```

Expected Output:

```
Access denied: invalid literal for int() with base 10: 'ab12'
Please remove your card
---
Access denied: Wrong PIN
Please remove your card
---
Welcome
Please remove your card
---
```

`finally` still runs when a function **returns** from `try`:

```
def demo():
    try:
        print("In try — about to return")
        return "success"
    finally:
        print("finally still runs before the return completes")

print("Result:", demo())
```

Expected Output:

```
In try — about to return
finally still runs before the return completes
Result: success
```

⚠️ WATCH OUT

```
# Weak — a later bug looks like a bad PIN
try:
    pin = int("1234")
    print("Welcome")          # if this fails, except may catch it
except ValueError:
    print("Invalid PIN")

# Prefer — success work in else
try:
    pin = int("1234")
except ValueError:
    print("Invalid PIN")
else:
    print("Welcome")
```

`else` does **not** run if `try` raised, even if `except` handled it. Do not `return` from `finally` — that can swallow the original exception.

Try It

For OTP values `"12ab"` and `"4321"`, `try` `int(otp)`, `else` print `OTP accepted`, `finally` print `sms slot released`. Treat anything other than `4321` as `ValueError("Wrong OTP")`.

---

5. `raise` — fail on purpose

Understand

You create an error on purpose when input or state is invalid. `raise ValueError("Amount must be positive")` stops the function and lets the **caller** decide how to recover (retry, show a form error, log).

After `raise`, the rest of that function does not run.

How It Works

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise ValueError("Not enough balance")
    return balance - amount

for amt in (200, -10, 5000):
    try:
        print("New balance:", withdraw(1000, amt))
    except ValueError as e:
        print("Withdraw failed:", e)
```

Expected Output:

```
New balance: 800
Withdraw failed: Amount must be positive
Withdraw failed: Not enough balance
```

⚠️ WATCH OUT

```
# Wrong — raise needs an exception object, not a bare string
raise "oops"

# Prefer
raise ValueError("Amount must be positive")
```

Raise a **specific** type — `ValueError`, or a custom class in the next section — not a generic `Exception` unless you truly have nothing more precise.

Try It

Write `book_seat(available, requested)` that raises `ValueError("Not enough seats")` when `requested > available`. Try `(5, 2)` and `(3, 5)`. Catch at the call site.

---

6. Custom exception class

Understand

A class you write that inherits from `Exception`, named for a **business** failure: `InsufficientBalanceError`, `InvalidAgeError`. Callers can `except InsufficientBalanceError` without treating every `ValueError` the same.

Empty `pass` is enough at first. Add `__init__` when you need extra fields (`account_id`, `roll`).

`raise` with **no** argument (bare `raise` inside `except`) re-raises the **same** exception so the caller can still handle it after you log.

How It Works

```
class InvalidRollError(Exception):
    def __init__(self, roll, message="Roll not found"):
        self.roll = roll
        super().__init__(f"{message}: {roll}")


def find_student(roll, db):
    if roll not in db:
        raise InvalidRollError(roll)
    return db[roll]


db = {101: "Anita", 102: "Ravi"}
for roll in (101, 999):
    try:
        print(find_student(roll, db))
    except InvalidRollError as e:
        print("Error:", e, "| roll attr:", e.roll)
```

Expected Output:

```
Anita
Error: Roll not found: 999 | roll attr: 999
```

Log, then let the caller handle:

```
def parse_marks(raw):
    try:
        return int(raw)
    except ValueError:
        print("ERROR: could not parse marks:", raw)
        raise


try:
    print("Marks:", parse_marks("eighty"))
except ValueError:
    print("Caller: show a form error, not a traceback")
```

Expected Output:

```
ERROR: could not parse marks: eighty
Caller: show a form error, not a traceback
```

⚠ WATCH OUT

```
# Wrong — not a subclass of Exception; callers will not catch it as expected
class InvalidRollError:
```

```
    pass

# Prefer
class InvalidRollError(Exception):
    pass
```

Try It

Define `OutOfStockError(Exception)`. `take_from_shelf(stock, qty)` raises it when `qty > stock`. Try qty `2` and `10` against stock `5`. Catch at the call site and print the error.

---

7. Common built-in exceptions

Understand

The type tells you *what* went wrong so the message can tell the user *what to do*.

Exception	Typical cause
`ValueError`	Wrong value (`int("abc")`)
`TypeError`	Wrong type (`"a" + 1`)
`IndexError`	Invalid list index
`KeyError`	Missing dict key
`FileNotFoundError`	File missing (File Handling uses this a lot)
`ZeroDivisionError`	`/` or `//` by zero
`AttributeError`	Missing attribute or method
`ImportError` / `ModuleNotFoundError`	Import failure

How It Works

Run each block. Print only the type name. (`except Exception` is for this quiz only — not a habit for real code.)

```
try:
    "a" + 1
except Exception as e:
    print(type(e).__name__)

try:
    [1][5]
except Exception as e:
    print(type(e).__name__)

try:
    {}["missing"]
except Exception as e:
    print(type(e).__name__)

try:
    1 // 0
except Exception as e:
    print(type(e).__name__)
```

```
try:
    int("hello")
except Exception as e:
    print(type(e).__name__)
```

Expected Output:

```
TypeError
IndexError
KeyError
ZeroDivisionError
ValueError
```

⚠️ WATCH OUT

Catch the type you can recover from. A quiz may use `except Exception` to print names. Production code names `ValueError`, `KeyError`, `FileNotFoundError`, and so on.

Try It

Without looking at the table, predict the type for `[][0]`, `int("")`, and `{1, 2}[0]`. Then run them in `try` / `except` and print `type(e).__name__`.

Step-by-Step: Build It

One assemble: `try` → `except` → `else` → `finally`. `finally` still runs when `except` or `else` **returns**.

```
def upi_transfer(pin, amount, balance):
    try:
        code = int(pin)
        if code != 4321:
            raise ValueError("Invalid PIN")
        if amount > balance:
            raise ValueError("Insufficient balance")
        new_balance = balance - amount
    except ValueError as e:
        print("Transfer failed:", e)
        return balance
    else:
        print(f"Sent ₹{amount}. New balance: ₹{new_balance}")
        return new_balance
    finally:
        print("Attempt logged")


print("Case 1 — bad PIN")
print("Returned:", upi_transfer("ab12", 200, 1000))
print()
print("Case 2 — success")
print("Returned:", upi_transfer("4321", 200, 1000))
print()
print("Case 3 — not enough money")
print("Returned:", upi_transfer("4321", 5000, 1000))
```

Expected Output:

```
Case 1 — bad PIN
Transfer failed: invalid literal for int() with base 10: 'ab12'
Attempt logged
Returned: 1000

Case 2 — success
Sent ₹200. New balance: ₹800
Attempt logged
Returned: 800

Case 3 — not enough money
Transfer failed: Insufficient balance
Attempt logged
Returned: 1000
```

Then open the kit: `18_safe_input_exceptions` — finish `read_age` in `starter/safe_age.py`. `InvalidAgeError` is already there. Do not copy `solution/`.

Practice Exercises

Attempt each problem before reading the model answer.

1. For each name, print the first letter. One name is `""` — catch `IndexError` so the loop continues.

Hint: `name[0]` raises `IndexError` on an empty string.

Model answer:

```
names = ["Priya", "", "Ravi"]
for name in names:
    try:
        print(name, "→", name[0])
    except IndexError:
        print("(blank name — skipped)")
```

Expected Output:

```
Priya → P
(blank name — skipped)
Ravi → R
```

2. Parse a price written as `"₹1,200"`. Strip `₹` and commas, then `int`. Bad strings raise `ValueError` at the call site.

Model answer:

```
def parse_inr(raw):
    cleaned = raw.replace("₹", "").replace(",", "").strip()
    return int(cleaned)


for raw in ("₹1,200", "n/a"):
    try:
        print(raw, "→", parse_inr(raw))
```

```
    except ValueError:
        print(raw, "→ not a price")
```

Expected Output:

```
₹1,200 → 1200
n/a → not a price
```

3. Raise custom `OutOfStockError` when requested qty is greater than stock. Catch it at the call site.

Model answer:

```
class OutOfStockError(Exception):
    pass


def take_from_shelf(stock, qty):
    if qty > stock:
        raise OutOfStockError(f"need {qty}, have {stock}")
    return stock - qty


for qty in (2, 10):
    try:
        print("Left:", take_from_shelf(5, qty))
    except OutOfStockError as e:
        print("Shelf:", e)
```

Expected Output:

```
Left: 3
Shelf: need 10, have 5
```

4. `try` `int(otp)`, `else` print accepted, `finally` print `"sms slot released"`. Use `"12ab"` and `"4321"`.

Model answer:

```
for otp in ("12ab", "4321"):
    try:
        code = int(otp)
        if code != 4321:
            raise ValueError("Wrong OTP")
    except ValueError as e:
        print("Denied:", e)
    else:
        print("OTP accepted")
    finally:
        print("sms slot released")
```

Expected Output:

```
Denied: invalid literal for int() with base 10: '12ab'
sms slot released
OTP accepted
sms slot released
```

5. `parse_qty` should log a line on `ValueError` then **re-raise**. The caller prints `show a form error`. Use input `"two"`.

Hint: Bare `raise` inside `except` re-raises the same exception.

Model answer:

```
def parse_qty(raw):
    try:
        return int(raw)
    except ValueError:
        print("ERROR: could not parse qty:", raw)
        raise


try:
    print("Qty:", parse_qty("two"))
except ValueError:
    print("show a form error")
```

Expected Output:

```
ERROR: could not parse qty: two
show a form error
```

Lab

Combine `try` / `except`, `raise`, a custom exception, `else`, and `finally`. This is not the age-input kit.

Problem

A fee desk processes a short list of raw amounts. Bad rows must not stop the batch. The clerk still needs a “row processed” line every time.

What to Build

A short script that walks three raw strings, accepts valid positive amounts, and prints a running total at the end.

Requirements

- Custom `InvalidAmountError(Exception)` — raise when the number is **≤ 0**. Message includes the bad value
- For each raw amount in `["1500", "abc", "-10"]`:
- `int(...)` in `try`
- Non-numeric → catch `ValueError` and print a clear message
- `<= 0` → raise / catch `InvalidAmountError`
- `else` → print `Accepted: <amount>` and add to a total

- `finally` → print `row processed`
- After the loop, print `Total: <sum of accepted amounts>`

Expected Behavior

`"abc"` does not dump a traceback. `"-10"` is an `InvalidAmountError`, not a silent skip. `finally` runs on every row, including failures.

Sample Run

```
Accepted: 1500
row processed
Need a number: invalid literal for int() with base 10: 'abc'
row processed
Invalid amount: -10
row processed
Total: 1500
```

(Your exact `ValueError` message may match Python's `int()` text. The **shape** should match.)

Constraints

You may use `def`. No type hints. No files. No logging module yet.

Bonus

If a row is valid but greater than `10000`, raise `InvalidAmountError` as "exceeds desk limit".

Mini-Project (Assignment): Safe Age Input CLI

Why this matters: Every form and CLI takes typed input. `"abc"` must not dump a traceback. Out-of-range values need a **named** business error, not a generic `ValueError` mixed with parse failures.

Lab kit: 15_Lab_Assets/18_safe_input_exceptions/

File	What you do
`starter/safe_age.py`	Finish `read_age`. `InvalidAgeError` is already declared. Keep the `__main__` loop
`broken/safe_age.py`	Bare `except:` returns `-1`; `"0"` looks successful. Fix both bugs
`solution/`	Do **not** copy this

This is **not** the UPI assemble from class. Same tools (`try` / `raise` / custom exception), different ticket.

Objective

Read an age from a string. Non-numeric input is `ValueError`. Age outside 1–120 is `InvalidAgeError`. Expected cases must not print a traceback.

Story / Scenario

Apollo Hospitals registration desk. A clerk types an age. `"abc"` must not dump a traceback on the counter screen. `"0"` and `"200"` are not valid patients. `"21"` must go through.

Requirements

Piece	Detail
`InvalidAgeError`	Subclass of `Exception`. Raise when the number is not in **1–120**. Message includes the bad value (`age out of range: 0`)
`read_age(raw)`	`int(raw)` in `try`. Non-numeric → `ValueError` with message `age must be an integer`. In range → return the `int`
Demo loop	Keep the four samples: `"21"`, `"abc"`, `"0"`, `"200"`. Print `sample`, then `->` and the age, **or** the exception type and message
Style	No type hints. No `raise ... from e`. No `assert`

Expected Behavior

- `"21"` prints `21 -> 21`
- `"abc"` is caught as `ValueError` — no traceback
- `"0"` and `"200"` are `InvalidAgeError`

Sample Run

```
21 -> 21
abc ValueError age must be an integer
0 InvalidAgeError age out of range: 0
200 InvalidAgeError age out of range: 200
```

Bonus Challenges

- Wrap `read_age` in a `while True` prompt until a valid age is entered
- Add `read_positive_float` for amounts (same pattern)
- **Later — Files week:** log failures with a timestamp

Done when

- [ ] `"21"` prints `21 -> 21`
- [ ] `"abc"` is caught as `ValueError` (`age must be an integer`) — no traceback
- [ ] `"0"` and `"200"` are `InvalidAgeError` (`age out of range: …`)
- [ ] `InvalidAgeError` inherits `Exception`
- [ ] `broken/` no longer uses bare `except:` and no longer accepts `0` / `200`

Interview Q&A

Q1. `try` / `except` / `else` / `finally` execution order?
Enter `try`. If a matching `except` fires, run that handler and **skip** `else`. If `try` raised nothing, run `else`. `finally` always runs after that (including when `try` / `except` / `else` returns). If nothing matched, `finally` still runs, then the traceback continues.

Q2. Why `else` instead of putting success code in `try`?
Success code in `try` can raise and be caught by `except`, so you report the wrong failure. `else` runs only after `try` fully succeeded.

Q3. `except Exception` vs bare `except:`?
`except Exception` catches application errors but not `KeyboardInterrupt` / `SystemExit`. Bare `except:` catches those too — hides bugs and can block Ctrl+C. Catch specific types.

Q4. When create a custom exception?
When callers must distinguish a domain failure (`InsufficientBalanceError`, `InvalidAgeError`) from a generic `ValueError`.

Q5. EAFP vs LBYL?
EAFP: try the operation, handle failure. LBYL: Look Before You Leap — check conditions first. Python culture favours EAFP for many I/O cases.

Q6. Exception vs `assert`?
`assert` is a developer sanity check (can be disabled with `-O`). Exceptions are for runtime errors and user-facing failures.

Q7. Why print or log before re-raising?
So the failure is recorded at the point it happened; `raise` then lets the caller decide the user-facing response. Catching and staying silent hides bugs.

Q8. Error vs exception in everyday language vs Python?
Everyday "error" covers syntax problems, runtime crashes, and wrong answers. In Python, **syntax errors** are parse-time (not for `try` / `except`), **exceptions** are runtime objects you can catch, and **logical errors** produce no traceback.

Cheat Sheet

```
try                 → risky attempt (keep it short)
except Type as e    → named recovery (first match wins)
except (A, B) as e  → same recovery for several types
else                → try succeeded (not swallowed by except)
finally             → always runs (cleanup / close / log)
raise ValueError()  → fail on purpose; caller recovers
raise               → re-raise the current exception
class E(Exception)  → custom business failure

# Never
except:             → also catches Ctrl+C
except: pass        → hides bugs
```

Syntax errors are not catchable in the same file. Logical errors produce no traceback — do not `except` them.